

MiniTienda

Registro y análisis de ventas — Evidencia de desarrollo

Documento con capturas de ejecución (entradas/salidas), explicación breve del algoritmo (ciclos y validaciones) y evidencia de cumplimiento de cada requisito del desafío, incluyendo los Retos A, B, C y D.

1. Descripción general del programa

MiniTienda es un programa de consola en Python que administra un catálogo de productos (tuplas), su precio y stock (diccionarios), registra ventas en un buffer (lista de diccionarios) que se vuelca a un DataFrame de pandas, persiste los datos en **ventas.csv**, calcula métricas con NumPy (media, desviación estándar y suma), agrupa ingresos por producto con *groupby* de pandas, y grafica dichos ingresos con Matplotlib. Toda la interacción con el usuario ocurre a través de un menú controlado por un bucle **while**.

1.1 Explicación del algoritmo (ciclos y validaciones)

El **menú principal** (`menu()`) es un bucle **while True** que se repite hasta que el usuario elige la opción 0. En cada iteración se valida que la opción ingresada pertenezca a un conjunto de opciones válidas; si no, se usa **continue** para volver a mostrar el menú sin procesar nada más. Dentro de cada opción se emplea **if/elif/else** para dirigir el flujo (ver catálogo, registrar venta, guardar/leer CSV, graficar, exportar PNG, agregar producto, salir).

La función `buscar_producto` recorre el catálogo (tupla de tuplas) con un **for**, usando **continue** para saltar los elementos que no coinciden con el id buscado y retornando en cuanto lo encuentra.

`registrar_venta` encadena validaciones con **if/elif/else**: producto inexistente, cantidad inválida (≤ 0) y stock insuficiente se rechazan explícitamente (y los dos primeros casos, junto con cualquier `producto_id` inexistente, se registran en **log.txt** — Reto D). Si todas las validaciones pasan, se calcula el descuento por volumen (Reto C, ≥ 10 unidades $\rightarrow$ 5%), se descuenta el stock y se agrega la venta al buffer (lista) y a la lista de IDs.

El manejo de errores usa **try/except/else/finally** en dos puntos clave: `leer_ventas_csv` (archivo inexistente o vacío) y `calcular_metricas` (división por cero controlada al calcular el ingreso por unidad vendida, cuando no hay unidades vendidas). Además, la conversión de la entrada de teclado a número usa **try/except ValueError**, y el cuerpo de cada opción del menú está envuelto en un **try/except Exception** general para que un error inesperado no cierre el programa.

2. Capturas de ejecución (entradas/salidas)

A continuación se muestra la salida real de la ejecución del script de pruebas (demo_test.py), que simula más de 10 ventas, incluyendo casos válidos e inválidos, y ejercita los cuatro retos.

```
#####
# 1) CATALOGO INICIAL
#####

=====
ID  PRODUCTO          CATEGORIA  PRECIO  STOCK
=====
1   Arroz 1kg          Abarrotes  $1.25   50
2   Aceite 1L          Abarrotes  $3.10   40
3   Leche 1L           Lacteos   $0.95   60
4   Pan Molde          Panaderia  $2.20   35
5   Detergente 1kg     Limpieza  $4.50   25
=====

#####
# 2) REGISTRO DE VENTAS (>= 10 ventas, incluye casos validos e invalidos)
#####
Venta registrada -> ID 1: 5 x Arroz 1kg = $6.25 (descuento $0.00)
Venta registrada -> ID 2: 3 x Aceite 1L = $9.30 (descuento $0.00)
Venta registrada -> ID 3: 12 x Leche 1L = $10.83 (descuento $0.57)
Venta registrada -> ID 4: 2 x Pan Molde = $4.40 (descuento $0.00)
Venta registrada -> ID 5: 1 x Detergente 1kg = $4.50 (descuento $0.00)
Venta registrada -> ID 6: 10 x Arroz 1kg = $11.88 (descuento $0.62)
Venta registrada -> ID 7: 4 x Aceite 1L = $12.40 (descuento $0.00)
Error: INTENTO FALLIDO: producto_id=99 no existe en el catalogo.
Venta registrada -> ID 8: 6 x Leche 1L = $5.70 (descuento $0.00)
Venta registrada -> ID 9: 15 x Pan Molde = $31.35 (descuento $1.65)
Venta registrada -> ID 10: 2 x Detergente 1kg = $9.00 (descuento $0.00)
Error: INTENTO FALLIDO: cantidad invalida (-3) para producto_id=2.
Venta registrada -> ID 11: 3 x Arroz 1kg = $3.75 (descuento $0.00)
Venta registrada -> ID 12: 4 x Leche 1L = $3.80 (descuento $0.00)

Total de ventas exitosas registradas: 12
IDs de venta (lista): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]

#####
# RETO A: agregar producto nuevo y actualizar precio/stock
#####
Producto agregado: (6, 'Yogurt 1L', 'Lacteos') | precio=$2.75 stock=30
Producto 6 actualizado -> precio=2.9, stock=28
Venta registrada -> ID 13: 11 x Yogurt 1L = $30.31 (descuento $1.59)
Venta registrada -> ID 14: 2 x Yogurt 1L = $5.80 (descuento $0.00)

=====
ID  PRODUCTO          CATEGORIA  PRECIO  STOCK
=====
1   Arroz 1kg          Abarrotes  $1.25   32
2   Aceite 1L          Abarrotes  $3.10   33
3   Leche 1L           Lacteos   $0.95   38
4   Pan Molde          Panaderia  $2.20   18
5   Detergente 1kg     Limpieza  $4.50   22
6   Yogurt 1L          Lacteos   $2.90   15
=====

#####
```

```

# 3) GUARDAR VENTAS EN CSV (Pandas)
#####
14 ventas guardadas en 'ventas.csv'.
id_venta  producto_id  producto_nombre  cantidad  precio_unitario  descuento  subtotal  fecha
1          1          Arroz 1kg         5          1.25          0.00        6.25 2026-08-20 02:49:36
2          2          Aceite 1L         3          3.10          0.00        9.30 2026-08-20 02:49:36
3          3          Leche 1L        12          0.95          0.57       10.83 2026-08-20 02:49:36
4          4          Pan Molde         2          2.20          0.00        4.40 2026-08-20 02:49:36
5          5  Detergente 1kg         1          4.50          0.00        4.50 2026-08-20 02:49:36
6          1          Arroz 1kg        10          1.25          0.62       11.88 2026-08-20 02:49:36
7          2          Aceite 1L         4          3.10          0.00       12.40 2026-08-20 02:49:36
8          3          Leche 1L         6          0.95          0.00        5.70 2026-08-20 02:49:36
9          4          Pan Molde        15          2.20          1.65       31.35 2026-08-20 02:49:36
10         5  Detergente 1kg         2          4.50          0.00        9.00 2026-08-20 02:49:36
11         1          Arroz 1kg         3          1.25          0.00        3.75 2026-08-20 02:49:36
12         3          Leche 1L         4          0.95          0.00        3.80 2026-08-20 02:49:36
13         6          Yogurt 1L        11          2.90          1.59       30.31 2026-08-20 02:49:36
14         6          Yogurt 1L         2          2.90          0.00        5.80 2026-08-20 02:49:36

#####
# 4) LEER VENTAS DESDE CSV (manejo de archivo inexistente)
#####
Error: No se encontro el archivo 'archivo_que_no_existe.csv'.
Fin del intento de lectura de ventas.csv.
Archivo 'ventas.csv' leido correctamente (14 filas).
Fin del intento de lectura de ventas.csv.

#####
# 5) METRICAS CON NUMPY + GROUPBY CON PANDAS
#####

--- Metricas generales (NumPy) ---
Ingreso total:      $149.27
Promedio por venta: $10.66
Desviacion estandar: $8.71
Ingreso por unidad: $1.87

--- Ingresos por producto (Pandas groupby) ---
Yogurt 1L          $36.11  (13 unidades)
Pan Molde          $35.75  (17 unidades)
Arroz 1kg          $21.88  (18 unidades)
Aceite 1L          $21.70  (7 unidades)
Leche 1L           $20.33  (22 unidades)
Detergente 1kg     $13.50  (3 unidades)

#####
# 6) GRAFICAR INGRESOS POR PRODUCTO (Matplotlib)
#####
Vista previa del grafico guardada en 'ingresos_preview.png'.

#####
# RETO B: Exportar grafico a PNG (opcion 6 del menu)
#####
Grafico exportado como 'ingresos.png'.

```

```
#####  
# RETO D: verificar que el intento fallido quedo en log.txt  
#####  
[2026-08-20 02:49:36] INTENTO FALLIDO: producto_id=99 no existe en el catalogo.  
[2026-08-20 02:49:36] INTENTO FALLIDO: cantidad invalida (-3) para producto_id=2.  
[2026-08-20 02:49:36] No se encontro el archivo 'archivo_que_no_existe.csv'.
```

DEMO COMPLETADA CORRECTAMENTE.

3. Archivo ventas.csv generado

El archivo contiene 14 ventas (se requerían al menos 10), generado con `pandas.DataFrame.to_csv`.

id_venta	producto_id	producto_nombre	cantidad	precio_unitario	descuento	subtotal	fecha
1	1	Arroz 1kg	5	1.25	0.0	6.25	2026-08-20 02:49:37
2	2	Aceite 1L	3	3.1	0.0	9.3	2026-08-20 02:49:37
3	3	Leche 1L	12	0.95	0.57	10.83	2026-08-20 02:49:37
4	4	Pan Molde	2	2.2	0.0	4.4	2026-08-20 02:49:37
5	5	Detergente 1kg	1	4.5	0.0	4.5	2026-08-20 02:49:37
6	1	Arroz 1kg	10	1.25	0.62	11.88	2026-08-20 02:49:37
7	2	Aceite 1L	4	3.1	0.0	12.4	2026-08-20 02:49:37
8	3	Leche 1L	6	0.95	0.0	5.7	2026-08-20 02:49:37
9	4	Pan Molde	15	2.2	1.65	31.35	2026-08-20 02:49:37
10	5	Detergente 1kg	2	4.5	0.0	9.0	2026-08-20 02:49:37
11	1	Arroz 1kg	3	1.25	0.0	3.75	2026-08-20 02:49:37
12	3	Leche 1L	4	0.95	0.0	3.8	2026-08-20 02:49:37
13	6	Yogurt 1L	11	2.9	1.59	30.31	2026-08-20 02:49:37
14	6	Yogurt 1L	2	2.9	0.0	5.8	2026-08-20 02:49:37

Archivo log.txt generado (Reto D)

```
[2026-08-20 02:49:36] INTENTO FALLIDO: producto_id=99 no existe en el catalogo.  
[2026-08-20 02:49:36] INTENTO FALLIDO: cantidad invalida (-3) para producto_id=2.  
[2026-08-20 02:49:36] No se encontro el archivo 'archivo_que_no_existe.csv'.  
[2026-08-20 02:49:37] INTENTO FALLIDO: producto_id=99 no existe en el catalogo.  
[2026-08-20 02:49:37] INTENTO FALLIDO: cantidad invalida (-3) para producto_id=2.  
[2026-08-20 02:49:37] No se encontro el archivo 'archivo_que_no_existe.csv'.  
[2026-08-20 02:49:37] Division por cero al calcular ticket promedio por unidad.
```

4. Gráfico de ingresos por producto (Matplotlib)

Gráfico de barras horizontales generado con Matplotlib y exportado con `plt.savefig("ingresos.png")` (Reto B, opción 6 del menú).

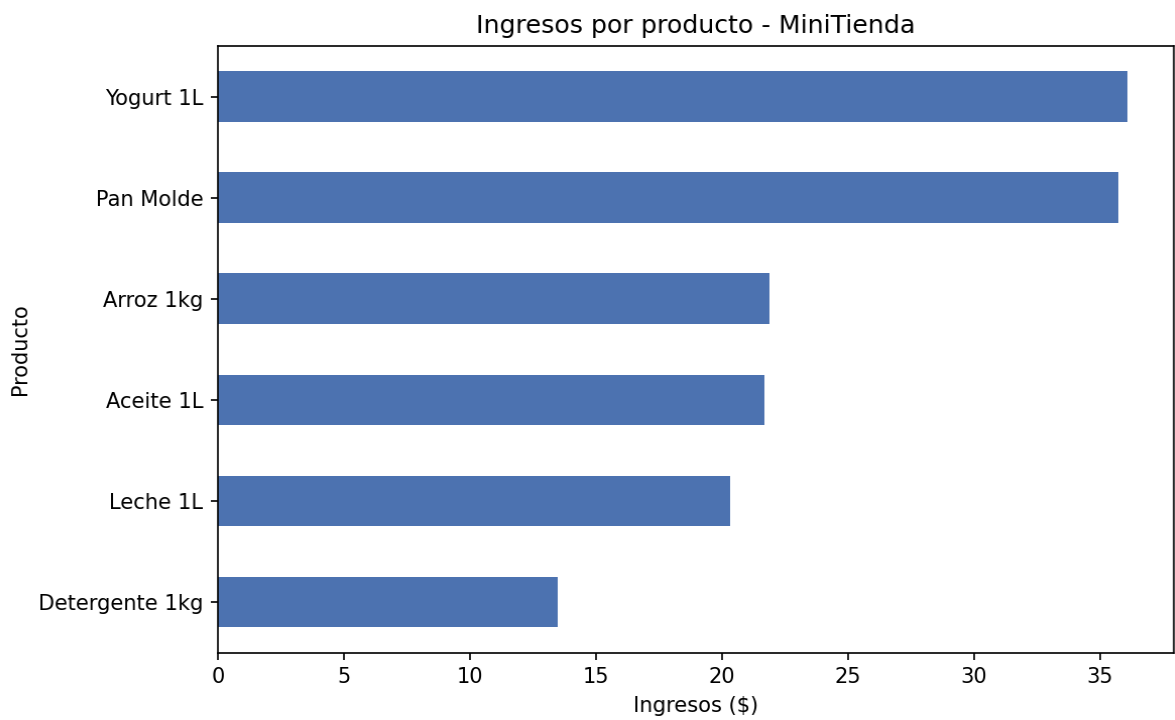


Figura 1. Ingresos por producto — MiniTienda.

5. Preguntas del entregable

¿Qué parte la hizo Pandas? ¿Qué parte NumPy?

Pandas construyó el DataFrame de ventas a partir de la lista de diccionarios, guardó y leyó el archivo CSV (`to_csv / read_csv`), y agrupó ingresos y unidades por producto con `groupby("producto_nombre").sum()`. **NumPy** calculó las métricas agregadas sobre el arreglo de subtotales: `np.mean`, `np.std`, `np.sum`, y la división protegida para el ingreso por unidad vendida.

¿Dónde usaste try/except y por qué?

En `leer_ventas_csv` (archivo inexistente o vacío, con `else/finally`); en `calcular_metricas` (división por cero controlada si no hay unidades vendidas); en el menú, al convertir texto a número (`ValueError`); y como red de seguridad general envolviendo cada opción del menú (`except Exception`) para que un error inesperado no cierre el programa abruptamente.

¿Qué estructuras son tuplas, listas y diccionarios en el código?

Tuplas: CATALOGO (tupla de tuplas `producto_id/nombre/categoría`) y cada producto individual devuelto por `buscar_producto`. **Listas:** `ventas` (buffer de diccionarios de venta) e `ids_venta` (lista de IDs). **Diccionarios:** `PRECIOS` (`id → precio`), `STOCK` (`id → cantidad`) y cada venta individual dentro de la lista `ventas`.

6. Checklist de cumplimiento de requisitos

Requisito	Cumplimiento
Tuplas: catálogo de productos	CATALOGO (tupla de tuplas) — Sección 1
Diccionarios: precios y stock	PRECIOS, STOCK — Sección 1
Listas: buffer de ventas / IDs	ventas, ids_venta — Sección 2
Funciones: todo modular	13 funciones en mini_tienda.py
Errores: input inválido / archivo inexistente / división por cero	try/except ValueError, FileNotFoundError, ZeroDivisionError — Secciones 2, 4 y 5
Archivos: ventas.csv + log.txt	Generados y adjuntos — Secciones 3 y 4
Pandas: DataFrame + groupby + CSV	guardar/leer_ventas_csv, calcular_metricas
NumPy: mean/std/sum sobre arreglos	calcular_metricas — Sección 5
Matplotlib: gráfica de barras	graficar_ingresos — Sección 4 (gráfico)
Control y bucles: if/elif/else, for, while, break, continue, try/except	control_usuario(), agregar_producto(), registrar_venta(), leer_ventas_csv()
CSV con al menos 10 ventas	14 ventas registradas — Sección 3
Reto A: agregar producto y actualizar precio/stock	agregar_producto, actualizar_precio_stock
Reto B: exportar gráfico a PNG (opción 6 del menú)	graficar_ingresos(exportar=True) → ingresos.png
Reto C: descuento 5% si unidades >= 10	calcular_descuento — visible en ventas ID 3, 6, 9, 13
Reto D: log de intentos fallidos con producto_id inexistente	escribir_log() dentro de registrar_venta